ROUND 1 DELIVERABLE • NAMMA SPACE COMPETITION

Code Repository & Setup Instructions

Raasta-3D: Complete Source Code & Free Cloud Pipeline Guide

Organized by **Techfest, IIT Bombay** in partnership with **Juspay**

> 📦 **Open-Source Code Repository**
>
> The complete codebase, documentation, pipeline notebooks, and sample metadata are hosted openly on GitHub:
>
> **GitHub Repository URL:** `https://github.com/Sarthakadhav/Raasta-3D`

1. System & Runtime Requirements

Component	Minimum Requirement	Recommended Specification
Web Browser	Any WebGL 2.0 compatible browser (Chrome, Edge, Firefox, Safari)	Latest Google Chrome or Microsoft Edge
Node.js Runtime	Node.js v18.0.0+	Node.js v20.x or v22.x LTS
Package Manager	npm v9+	npm, pnpm, or yarn
Hardware (Client)	Standard integrated graphics (Intel UHD / Iris Xe, AMD Radeon)	Dedicated GPU for sustained 60 FPS rendering
Cloud GPU (Pipeline)	Google Colab Free Tier (1x NVIDIA T4 GPU, 15GB VRAM)	Zero local GPU required; training is 100% free

2. Step-by-Step Local Setup & Execution

Step 1: Clone the GitHub Repository

```
git clone https://github.com/Sarthakadhav/Raasta-3D.git
cd Raasta-3D
```

Step 2: Install Project Dependencies

```
npm install
```

Step 3: Launch Local Development Server

```
npm run dev
```

The application will launch with hot module replacement (HMR) and can be accessed at:

```
http://localhost:5173
```

Step 4: Production Build & Preview

To compile the production distribution bundle and preview it locally:

```
npm run build
npm run preview
```

3. Project Architecture & Directory Structure

```
Raasta---3d/
├── src/
│   ├── components/              # Modular React UI & 3D HUD Components
│   │   ├── SplatViewer.tsx      # Dual WebGL Three.js + GaussianSplats3D Canvas
│   │   ├── NavigationHUD.tsx    # Turn-by-turn guidance cues & auto-roam controller
│   │   ├── Minimap.tsx          # Real-time 2D radar minimap with vision cone
│   │   ├── AROverlay.tsx        # Simulated mobile AR HUD with 360° compass tape
│   │   ├── RoomDetailsCard.tsx  # Dynamic room zone title, square footage & items
│   │   ├── POIDetailsCard.tsx   # Landmark inspection & route request trigger
│   │   ├── POISearchModal.tsx   # Fuzzy search command palette (Ctrl+K)
│   │   └── UploadModal.tsx      # In-browser video grinding & .splat loader
│   ├── data/
│   │   ├── homeDataset.ts       # Round 1 Smart Home dataset (POIs, zones, navmesh)
│   │   └── officeDataset.ts     # Bonus Juspay Innovation Hub corporate dataset
│   ├── types/
│   │   └── spatial.ts           # TypeScript interfaces for POIs, zones, AABB, nav
│   └── utils/
│       ├── navGraph.ts          # Non-colliding A* & string-pulling funnel engine
│       ├── indoorSceneBuilder.ts# CAD walls, furniture suites, downlights & beacons
│       └── videoTo3DConverter.ts# Client-side video keyframe & 3DGS synthesis
├── pipeline/
│   └── Namma_Space_Raasta3D_Pipeline.ipynb # 100% Free Google Colab T4 Notebook
├── submission/                  # Round 1 Submission Documentation & PDFs
└── test/                        # Sensor metadata, capture telemetry & tests
```

4. 100% Free Cloud Photogrammetry Pipeline (Google Colab)

Evaluators wishing to train a custom 3D Gaussian Splat from their own smartphone video can run our automated Colab notebook with zero setup:

1. Open `pipeline/Namma_Space_Raasta3D_Pipeline.ipynb` in **Google Colab**.
2. Select **Runtime > Change runtime type > Hardware accelerator > T4 GPU** (100% free).
3. Upload your smartphone video (e.g., `indoor_capture.mp4`).
4. Execute all cells sequentially:
   - `ns-process-data video`: Decodes frames and executes COLMAP Structure-from-Motion.
   - `ns-train splatfacto`: Optimizes 3D Gaussian Splats in ~13.8 minutes.
   - `ns-export gaussian-splat`: Generates the compressed `.ply` / `.splat` binary.
5. Download the resulting `.splat` file and drag it directly into the Raasta-3D web viewer via the **"Load Splat"** button.

5. Verification & Code Quality

The codebase is strictly typed and verified with zero compilation errors:

```
npm run lint    # ESLint verification (0 errors, 0 warnings)
npx tsc --noEmit# TypeScript type verification (0 errors)
npm run build   # Production Vite compilation (Success in 1.07s)
```